

Netflix EDA

In [1]:

```
# importing lib
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

In [2]:

```
df = pd.read_csv('mymoviedb.csv', lineterminator='\n')
```

In [3]:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9827 entries, 0 to 9826
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Release_Date          9827 non-null   object
1   Title                 9827 non-null   object
2   Overview              9827 non-null   object
3   Popularity            9827 non-null   float64
4   Vote_Count            9827 non-null   int64
5   Vote_Average          9827 non-null   float64
6   Original_Language     9827 non-null   object
7   Genre                 9827 non-null   object
8   Poster_Url            9827 non-null   object
dtypes: float64(2), int64(1), object(6)
memory usage: 691.1+ KB
```

- looks like our dataset has no NaNs!
- Overview, Original_Language and Poster-Url wouldn't be so useful during analysis
- Release_Date column needs to be casted into date time and to extract only the year value

In [4]:

```
df['Genre'].head()
```

Out[4]:

```
0   Action, Adventure, Science Fiction
1           Crime, Mystery, Thriller
2           Thriller
3   Animation, Comedy, Family, Fantasy
4   Action, Adventure, Thriller, War
Name: Genre, dtype: object
```

In [5]:

```
df.duplicated().sum()
```

Out[5]:

```
np.int64(0)
```

In [6]:

```
df.isnull().sum()
```

Out[6]:

Release_Date 0
Title 0
Overview 0
Popularity 0
Vote_Count 0
Vote_Average 0
Original_Language 0
Genre 0
Poster_Url 0
dtype: int64

In [7]:

```
df.describe()
```

Out[7]:

	Popularity	Vote_Count	Vote_Average
count	9827.000000	9827.000000	9827.000000
mean	40.326088	1392.805536	6.439534
std	108.873998	2611.206907	1.129759
min	13.354000	0.000000	0.000000
25%	16.128500	146.000000	5.900000
50%	21.199000	444.000000	6.500000
75%	35.191500	1376.000000	7.100000
max	5083.954000	31077.000000	10.000000

In [8]:

```
## Data Explorartion
### Data Structure: The dataset currently consists of 9,827 rows and 9 columns.
### Data Integrity: There are currently no duplicate or null values in the dataset.
### Release Date Formatting: The release_date column needs to be converted into a formal
### Column Removal: Three columns—overview, original_language, and poster_url—are consid
### Data Categorization: The vote_average column requires categorical labeling to better
### Cleaning Genres: The genre column contains white spaces that need to be removed to e
```

In [9]:

```
df.head()
```

Out[9]:

	Release_Date	Title	Overview	Popularity	Vote_Count	Vote_Average	Original_Language	
0	2021-12-15	Spider-Man: No Way Home	Peter Parker is unmasked and no longer able to...	5083.954	8940	8.3	en	A
1	2022-03-01	The Batman	In his second year of fighting crime, Batman u...	3827.658	1151	8.1	en	(
2	2022-02-25	No Exit	Stranded at a rest stop in the	2618.087	122	6.3	en	T

	Release_Date	Title	Overview	Popularity	Vote_Count	Vote_Average	Original_Language	
			mountains durin...					
3	2021-11-24	Encanto	The tale of an extraordinary family, the Madri...	2402.201	5076	7.7	en	Anim Co F Fa
4	2021-12-22	The King's Man	As a collection of history's worst tyrants and...	1895.511	1793	7.0	en	A Adve T

In [10]:

```
df['Release_Date']=pd.to_datetime (df['Release_Date'])
```

In [11]:

```
print(df['Release_Date'].dtype)
```

datetime64[ns]

In [12]:

```
df['Release_Date'] = df ['Release_Date'].dt.year
```

In [13]:

```
df.head()
```

Out[13]:

	Release_Date	Title	Overview	Popularity	Vote_Count	Vote_Average	Original_Language	
0	2021	Spider-Man: No Way Home	Peter Parker is unmasked and no longer able to...	5083.954	8940	8.3	en	A Adve Sc F
1	2022	The Batman	In his second year of fighting crime, Batman u...	3827.658	1151	8.1	en	(My T
2	2022	No Exit	Stranded at a rest stop in the mountains durin...	2618.087	122	6.3	en	T
3	2021	Encanto	The tale of an extraordinary family, the Madri...	2402.201	5076	7.7	en	Anim Co F Fa
4	2021	The King's Man	As a collection of history's worst tyrants and...	1895.511	1793	7.0	en	A Adve T

Dropping Overview, Original_Language and Poster-Url

In [14]:

```
cols = ['Overview', 'Original_Language', 'Poster_Url']
```

In [15]:

```
df.drop(cols, axis = 1, inplace = True )  
df.columns
```

Out[15]:

```
Index(['Release_Date', 'Title', 'Popularity', 'Vote_Count', 'Vote_Average',  
      'Genre'],  
      dtype='object')
```

In [16]:

```
df.head()
```

Out[16]:

	Release_Date	Title	Popularity	Vote_Count	Vote_Average	Genre
0	2021	Spider-Man: No Way Home	5083.954	8940	8.3	Action, Adventure, Science Fiction
1	2022	The Batman	3827.658	1151	8.1	Crime, Mystery, Thriller
2	2022	No Exit	2618.087	122	6.3	Thriller
3	2021	Encanto	2402.201	5076	7.7	Animation, Comedy, Family, Fantasy
4	2021	The King's Man	1895.511	1793	7.0	Action, Adventure, Thriller, War

In [17]:

```
def categorize_col(df, col, labels):  
    edges = [df[col].describe()['min'],  
             df[col].describe()['25%'],  
             df[col].describe()['50%'],  
             df[col].describe()['75%'],  
             df[col].describe()['max']]  
  
    df[col] = pd.cut(df[col], edges, labels = labels, duplicates = 'drop')  
    return df
```

In [18]:

```
labels = ['not_popular', 'below_avg', 'average', 'popular']  
categorize_col(df, 'Vote_Average', labels)  
  
df['Vote_Average'].unique()
```

Out[18]:

```
['popular', 'below_avg', 'average', 'not_popular', NaN]  
Categories (4, object): ['not_popular' < 'below_avg' < 'average' < 'popular']
```

In [19]:

```
df.head()
```

Out[19]:

	Release_Date	Title	Popularity	Vote_Count	Vote_Average	Genre
0	2021	Spider-Man: No Way Home	5083.954	8940	popular	Action, Adventure, Science Fiction
1	2022	The Batman	3827.658	1151	popular	Crime, Mystery, Thriller
2	2022	No Exit	2618.087	122	below_avg	Thriller
3	2021	Encanto	2402.201	5076	popular	Animation, Comedy, Family, Fantasy
4	2021	The King's Man	1895.511	1793	average	Action, Adventure, Thriller, War

In [20]:

```
df['Vote_Average'].value_counts()
```

Out[20]:

```
Vote_Average
not_popular    2467
popular        2450
average        2412
below_avg      2398
Name: count, dtype: int64
```

In [21]:

```
df.dropna(inplace = True)

df.isna().sum()
```

Out[21]:

```
Release_Date    0
Title           0
Popularity       0
Vote_Count      0
Vote_Average    0
Genre           0
dtype: int64
```

In [22]:

```
df.head()
```

Out[22]:

	Release_Date	Title	Popularity	Vote_Count	Vote_Average	Genre
0	2021	Spider-Man: No Way Home	5083.954	8940	popular	Action, Adventure, Science Fiction
1	2022	The Batman	3827.658	1151	popular	Crime, Mystery, Thriller
2	2022	No Exit	2618.087	122	below_avg	Thriller
3	2021	Encanto	2402.201	5076	popular	Animation, Comedy, Family, Fantasy
4	2021	The King's Man	1895.511	1793	average	Action, Adventure, Thriller, War

we'd split genres into a list and then

explode our dataframe to have only onedf =
`df.dropna(subset=['Genre'])`

```
df['Genre'] = df['Genre'].str.split(', ') df = df.explode('Genre').reset_index(drop=True)
```

genre per row for each movie

In [23]:

```
df = df.dropna(subset=['Genre'])

df['Genre'] = df['Genre'].str.split(', ')
df = df.explode('Genre').reset_index(drop=True)
```

In [25]:

```
df.head()
```

Out[25]:

	Release_Date	Title	Popularity	Vote_Count	Vote_Average	Genre
0	2021	Spider-Man: No Way Home	5083.954	8940	popular	Action
1	2021	Spider-Man: No Way Home	5083.954	8940	popular	Adventure
2	2021	Spider-Man: No Way Home	5083.954	8940	popular	Science Fiction
3	2022	The Batman	3827.658	1151	popular	Crime
4	2022	The Batman	3827.658	1151	popular	Mystery

In [26]:

```
# casting column into category
df['Genre'] = df['Genre'].astype('category')
# confirming changes
df['Genre'].dtypes
```

Out[26]:

```
CategoricalDtype(categories=['Action', 'Adventure', 'Animation', 'Comedy', 'Crime',
                             'Documentary', 'Drama', 'Family', 'Fantasy', 'History',
                             'Horror', 'Music', 'Mystery', 'Romance', 'Science Fiction',
                             'TV Movie', 'Thriller', 'War', 'Western'],
                  ordered=False, categories_dtype=object)
```

In [27]:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 25552 entries, 0 to 25551
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Release_Date    25552 non-null  int32
```

```
1 Title          25552 non-null object
2 Popularity     25552 non-null float64
3 Vote_Count     25552 non-null int64
4 Vote_Average   25552 non-null category
5 Genre          25552 non-null category
dtypes: category(2), float64(1), int32(1), int64(1), object(1)
memory usage: 749.6+ KB
```

In [28]:

```
df.nunique()
```

Out[28]:

```
Release_Date    100
Title           9415
Popularity      8088
Vote_Count      3265
Vote_Average     4
Genre           19
dtype: int64
```

Data Visualization

In [29]:

```
# setting up seaborn configurations
sns.set_style('whitegrid')
```

Q1: What is the most frequent genre in the dataset?

In [30]:

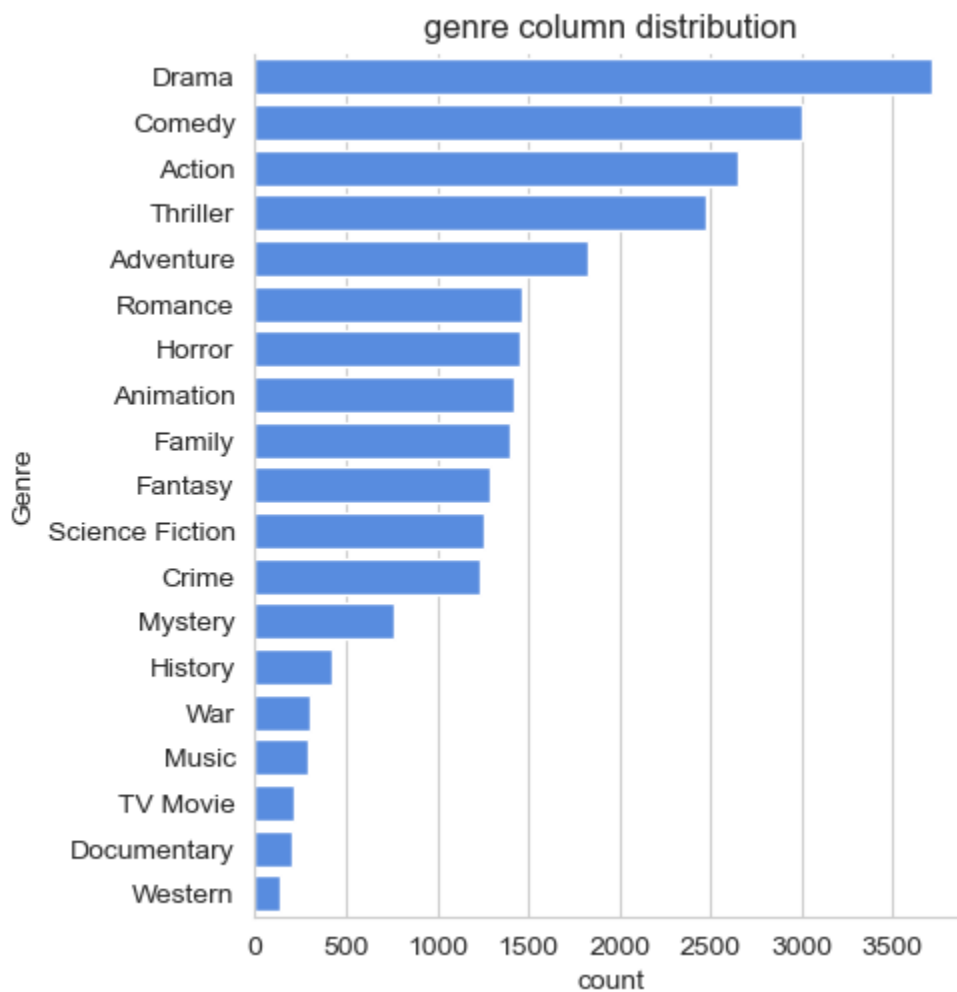
```
# showing stats. on genre column
df['Genre'].describe()
```

Out[30]:

```
count    25552
unique      19
top      Drama
freq     3715
Name: Genre, dtype: object
```

In [31]:

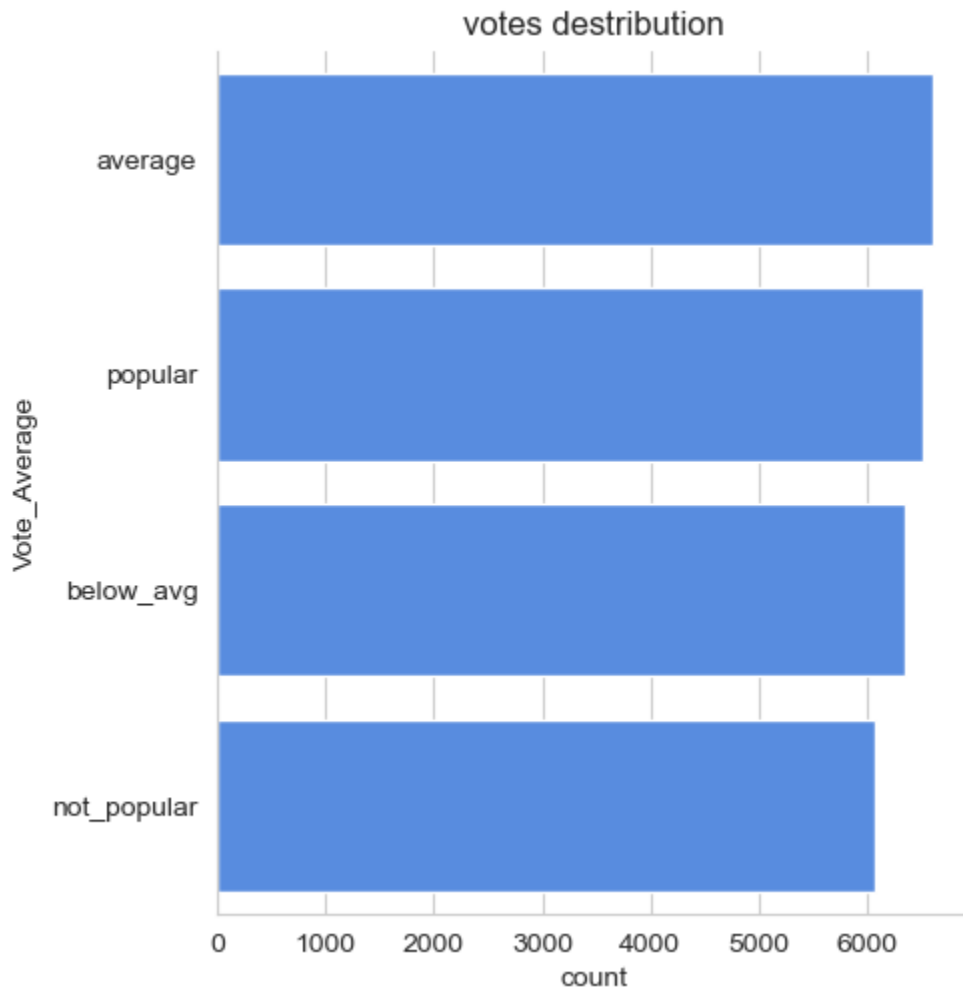
```
# visualizing genre column
sns.catplot(y = 'Genre', data = df, kind = 'count',
order = df['Genre'].value_counts().index,
color = '#4287f5')
plt.title('genre column distribution')
plt.show()
```



Q2: What genres has highest votes ?

In [32]:

```
# visualizing vote_average column
sns.catplot(y = 'Vote_Average', data = df, kind = 'count',
order = df['Vote_Average'].value_counts().index,
color = '#4287f5')
plt.title('votes distribution')
plt.show()
```

Q3: What movie got the highest popularity ? what's its genre ?

In [33]:

```
# checking max popularity in dataset
df[df['Popularity'] == df['Popularity'].max()]
```

Out[33]:

	Release_Date	Title	Popularity	Vote_Count	Vote_Average	Genre
0	2021	Spider-Man: No Way Home	5083.954	8940	popular	Action
1	2021	Spider-Man: No Way Home	5083.954	8940	popular	Adventure
2	2021	Spider-Man: No Way Home	5083.954	8940	popular	Science Fiction

Q4: What movie got the lowest popularity? what's its genre?

In [34]:

```
# checking max popularity in dataset
df[df['Popularity'] == df['Popularity'].min()]
```

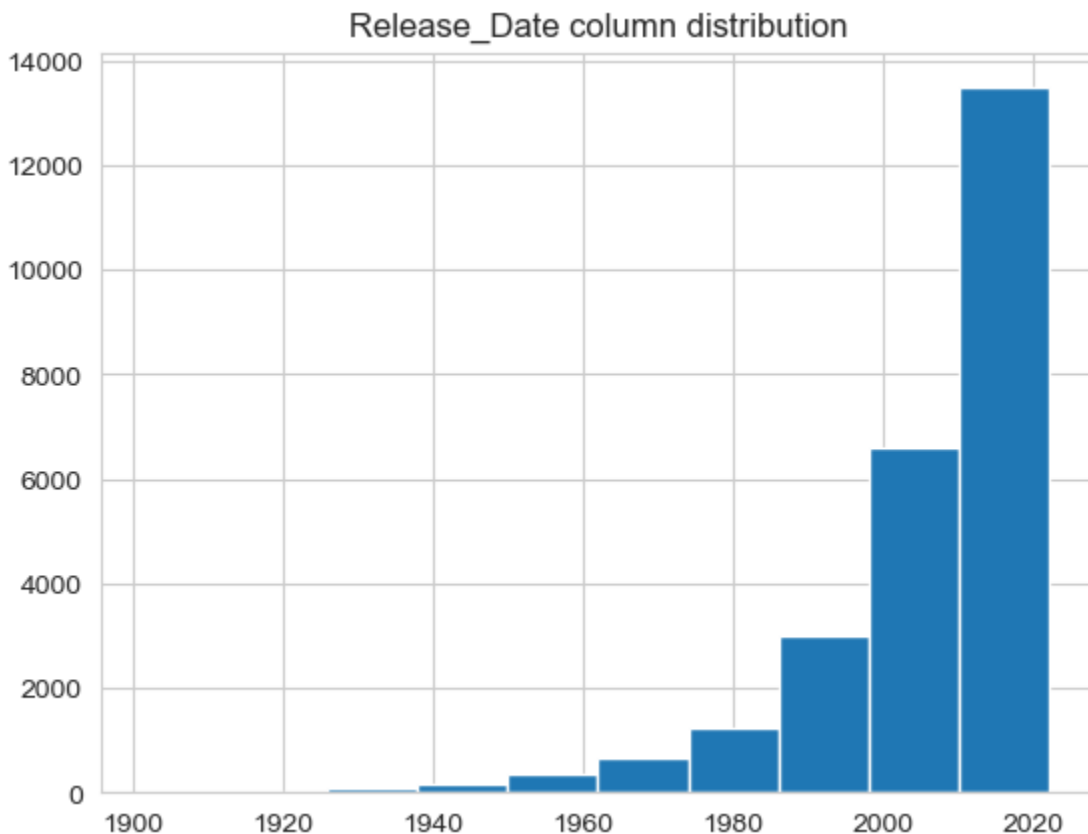
Out[34]:

	Release_Date	Title	Popularity	Vote_Count	Vote_Average	Genre
25546	2021	The United States vs. Billie Holiday	13.354	152	average	Music
25547	2021	The United States vs. Billie Holiday	13.354	152	average	Drama
25548	2021	The United States vs. Billie Holiday	13.354	152	average	History
25549	1984	Threads	13.354	186	popular	War
25550	1984	Threads	13.354	186	popular	Drama
25551	1984	Threads	13.354	186	popular	Science Fiction

Q5: Which year has the most filmed movies?

In [35]:

```
df['Release_Date'].hist()  
plt.title('Release_Date column distribution')  
plt.show()
```



Conclusion

Q1: What is the most frequent genre in the dataset?

Drama genre is the most frequent genre in our dataset and has appeared more than 14% of the times among 19 other genres.

Q2: What genres has highest votes ?

we have 25.5% of our dataset with popular vote (6520 rows). Drama again gets the highest popularity among fans by being having more than 18.5% of movies popularities.

Q3: What movie got the highest popularity ? what's its genre ?

Spider-Man: No Way Home has the highest popularity rate in our dataset and it has genres of Action , Adventure and Sience Fiction .

Q3: What movie got the lowest popularity ? what's its genre ?

The united states, thread' has the highest lowest rate in our dataset and it has genres of music , drama , 'war', 'sci-fi' and history`.

Q4: Which year has the most filmed movies?

year 2020 has the highest filmming rate in our dataset

In []: